B16 PyTorch hygiene

Device handling, training loops and reproducibility.

B16-L1 Choose the device once, move model and data to it

Pick the device once and pass it in. Reassign tensors after .to(). Never hardcode "cuda".

```python
def pick_device(prefer_gpu: bool = True) -> torch.device:
    if prefer_gpu and torch.cuda.is_available():
        return torch.device("cuda")
    if prefer_gpu and torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")
```

Choose a device	Once, at the edge: cuda if available, then mps, then cpu. Pass it in.
Tensor.to vs Module.to	Tensor.to returns a new tensor (reassign). Module.to moves in place.
GPU tensor to NumPy	t.detach().cpu().numpy().
Hardcoded .cuda()	Breaks on CPU machines and CI. Use .to(device).

B16-L2 A training loop that does what you think

zero_grad, train and eval modes, inference_mode for validation, logits into CrossEntropyLoss, loss.item() for logs, and early stopping on validation loss.

```python
for epoch in range(max_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
    val_loss = evaluate(model, val_loader, loss_fn, device)   # model.eval() + torch.inference_mode()
```

The training iteration	to(device), zero_grad, forward and loss, backward, step.
Validation mode	model.eval() and torch.inference_mode(). Back to model.train() after.
CrossEntropyLoss input	Raw logits. It applies log-softmax itself.
Early stopping	Stop when validation loss has not improved by min_delta for patience epochs. Keep the best weights.

B16-L3 Reproducible PyTorch runs and safe checkpoints

Seed every generator at the entry point, give DataLoaders their own generator, and checkpoint state dicts, not objects.

```python
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)                          # CPU and all CUDA devices
loader_gen = torch.Generator().manual_seed(seed)
train_loader = DataLoader(ds, batch_size=64, shuffle=True, generator=loader_gen)
```

```python
torch.save({"model": model.state_dict(), "optimizer": optimizer.state_dict(),
            "epoch": epoch, "torch_rng": torch.get_rng_state()}, path)
```

Seed a PyTorch run	random.seed, np.random.seed, torch.manual_seed at the entry point, plus a seeded Generator for each DataLoader.

torch.load default in PyTorch 2.14	Weights only. Pickled objects fail with UnpicklingError.
Checkpoint contents	Model and optimizer state dicts, epoch, RNG state, best metric.
Deterministic operations	torch.use_deterministic_algorithms(True) raises on operations with no deterministic version.